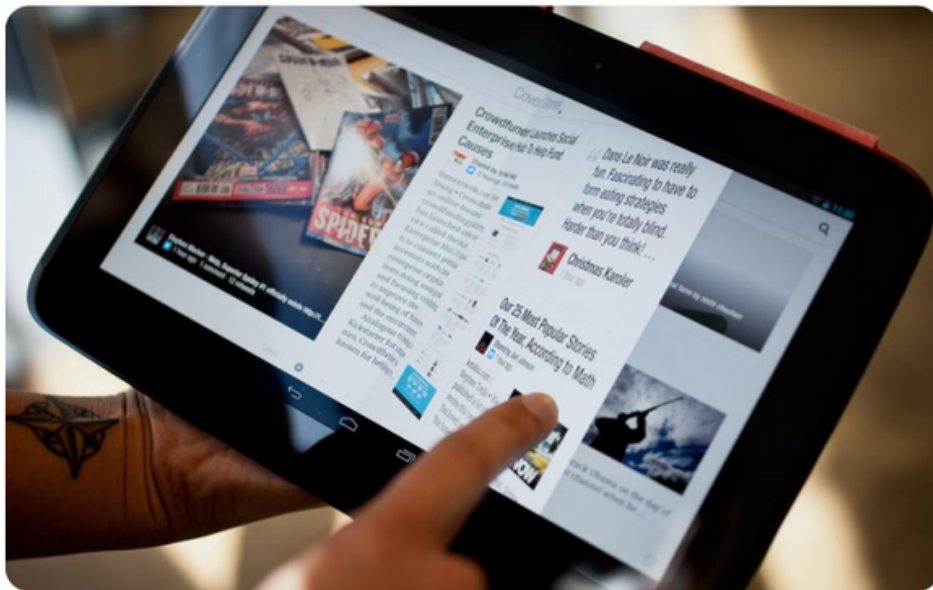


Specification document

EpiFlipboard



Gaël EDMOND - Jerome PATON

{EPITECH}

Project presentation

Context

Pro Project is a project that aims to recode an already existing software and imagine the development process of a project in a real company.

Objective

The goal of EpiFlipBoard is to build an application that allows users to:

- Read news articles from multiple external sources (APIs, RSS, etc.)
- View articles in a visual, magazine-style layout
- Filter articles by category or source
- Search for specific topics or keywords
- Read full articles directly inside the app
- (Optional) Save or share favorite articles

The app must be intuitive, visually appealing, and functional offline (if possible), while maintaining a modular and maintainable architecture.

Target audience

- General users who wish to read and organize multiple news sources in one app.
- Students, journalists, and professionals who value a clean, modern reading experience.
- Users who prefer an alternative to web-based news readers.

Functionnnalities

Feature	Description	Priority
Authentication	Allow users to register/login and fetch data on multiple devices	High
Home Feed	Display news articles from an API or RSS feed	High
Magazine Layout	Card view with image, title, description	High
Article Reader	View the full article content in-app	High
Category Filtering	Filter articles by topic or source	High
Search Bar	Search for keywords or topics	Medium
Offline Access (Optional)	Cache articles for offline reading	Medium
Save/Bookmark Articles	Allow users to store favorites locally	Medium
Light/Dark Mode	Improve visual comfort	Low
High contrast / Font size	Accessibility features	Low

Technical constraints

- The app should be optimized for most recent android versions
- The app's response time should be fast (less than 3 seconds)
- The app's content should be updated when the app is not running

Technical Architecture:

- Language: Dart
- Front-end: Flutter
- Back-end: Python (FastAPI)
- External API: NewsAPI / RSS feeds / Reddit API
- Database: PostgreSQL
- CI/CD: GitHub Actions
- Containerization: Docker / Docker Compose

User Interface and Design System

- Language: Dart
- Framework: Flutter UI Toolkit
- Design Style: Material Design 3 (Google guidelines)
- Color Themes: Light and Dark
- Navigation: Bottom navigation bar + category tabs
- Animation: Flutter's Hero animations + AnimatedContainer
- Responsiveness: Adaptive layouts for mobile and desktop
- Fonts and Icons: Google Fonts + Material Icons

UI Tools:

- Figma → for wireframes and design mockups
- Flutter DevTools → for debugging UI and performance

DevOps Environment

- Git + GitHub
 - GitHub repository for code and CI/CD pipelines
 - Branching model: GitFlow (main / develop / feature branches)
 - GitHub Projects for Kanban task tracking
 - GitHub Issues for bug tracking

CI/CD:

- Tool: GitHub Actions
- Pipeline Configuration:
 - Lint and format check (Dart analyzer)
 - Run unit and widget tests
 - Build the app (Flutter build)
 - Generate documentation (Markdown → PDF)
 - Package the app in Docker image for deployment or demo
- Artifacts: Compiled .apk, .exe, or .ApplImage depending on target platform.

Docker Environment:

- Goal: Ensure identical build and run environments for all developers and CI servers.
- Setup:
- Base image: cirrusci/flutter or google/dart
- Dockerfile includes dependencies for:
 - Flutter SDK installation
 - Build tools (Android SDK / Linux tools)
 - Running tests
- Docker Compose: optional for adding mock API server during testing.

Distribution:

- Via GitHub Releases or internal delivery.
- Future possibility: automated publishing using Fastlane or Codemagic.

Project Management

- The projects deadline is set on 28/01/2026
- The project is developped using Agile method with team meeting at the start and the end of each sprint
- During development, the project will be separated between multiple obtainable milestones
- Communication tools : Teams / Discord
- Project management tools : GitHub Projects

Period	Objective	Deliverables
Oct 20 – Oct 30	Benchmark, requirements,	Benchmark + Specification
Nov 3 – Nov 20	UI/UX design and architecture setup	Mockups + technical
Nov 24 – Dec 18	MVP development (feed display,	Alpha version
Jan 6 – Jan 20	Add filters, search, and offline mode	Beta version
Jan 21 – Jan 28	Finalization, Docker setup,	Final version + presentation